Smart Camera &

Video Surveillance System

New Features Documentation

Object Detection (YOLO) + Web Dashboard

Date: 04 October 2026

Phase: Feature Expansion (80% Complete)

Status: Implemented & Tested

Final Year B.Tech (CSE) Major Project

Table of Contents

1. Introduction - What Was Added
2. What is YOLO Object Detection?
3. Object Detector Module (object_detector.py)
4. Web Dashboard - How It Works
5. Dashboard Video Processor (dashboard.py)
6. Flask Web App (app.py)
7. HTML Templates (base, index, alerts, known_faces)
8. How All Modules Work Together
9. How to Run the New Features
10. What You See on Screen
11. Files in This Update
12. Troubleshooting

1. Introduction - What Was Added

Two major features were added on top of the existing 60% surveillance system:

FEATURE A - OBJECT DETECTION WITH YOLO: The camera can now detect not just faces, but also objects (people, bags, cell phones, suitcases, etc.) in the frame. Suspicious objects (backpack, handbag, suitcase, cell phone) are highlighted in red with a warning status.

FEATURE B - WEB DASHBOARD: The system now runs as a web application. You open a browser, see the live camera feed with all detections drawn on it in real-time, view detection history, registered faces, and statistics - all from any device connected to the same network.

2. What is YOLO Object Detection?

YOLO stands for "You Only Look Once". It is a famous deep learning algorithm for detecting objects in images. Unlike older methods that scanned an image piece-by-piece, YOLO looks at the whole image ONCE (in a single pass) and directly predicts: what objects are there, where they are, and how confident it is.

How YOLO Works (Simple Explanation)

1. The image is divided into a grid (e.g., 13 x 13 cells).
2. Each cell predicts a few bounding boxes and a confidence score.
3. Each box gets a class label from 80 known classes (person, car, dog, backpack...).
4. NMS (Non-Maximum Suppression) removes duplicate boxes around the same object.
5. Final output: list of (label, confidence, box) for each object found.

YOLOv4-tiny - The Model We Use

We use a lightweight version called YOLOv4-tiny. It is small (~24MB) and fast, so it can run in real-time on a normal laptop CPU without a graphics card. It still knows all 80 common object classes.

How We Run It (OpenCV DNN)

We do NOT run YOLO directly in Python. Instead, OpenCV has a module called "dnn" (Deep Neural Network) that can load YOLO model files and run them. This avoids installing heavy deep-learning frameworks.

File	Purpose
yolov4-tiny.cfg	Network architecture (how many layers, what sizes)
yolov4-tiny.weights	The trained weights (the learned knowledge)
coco.names	List of 80 class names that the model knows

3. Object Detector Module (src/object_detector.py)

Purpose

This module loads the YOLO model and provides reusable functions to detect and draw objects on any image or video frame.

Class: ObjectDetector

It has 6 methods. Here is what each one does:

__init__(self)
Constructor. Calls three helper methods to load the network, load class names, and compute the output layer names. Stored as self.net, self.classes, self.output_layers.

_load_network(self)
Checks that the .cfg and .weights files exist. Then calls cv2.dnn.readNetFromDarknet() to create the network object. Sets backend to OpenCV and target to CPU so it works on any laptop.

_load_classes(self)
Reads coco.names line by line and stores the 80 class names in a list.

_get_boxes(self, outputs, width, height)
This is the "decoder". YOLO gives raw numbers; this method translates them into boxes. For each detection it picks the class with the highest score, filters out low-confidence ones (below YOLO_CONFIDENCE=0.4), converts center-point coordinates into x,y,width,height, and finally runs Non-Maximum-Suppression (NMSBoxes) to remove overlapping boxes for the same object.

detect_objects(self, frame)
The main function. Converts the frame into a "blob" (a standardized numerical format the model expects: resized to 416x416, scaled to 0-1). Feeds it to the network with setInput(), runs forward(), decodes with _get_boxes(), and returns a list of dictionaries: {label, confidence, box, is_suspicious}.

draw_objects(self, frame, detections)
Draws a colored rectangle + label for each object. Suspicious objects are RED, normal objects are BLUE. Returns the annotated frame.

count_suspicious(self, detections)
Counts how many suspicious objects were found in the current frame.

Important Settings (config/settings.py)

```
YOLO_CFG         = models/yolov4-tiny.cfg
YOLO_WEIGHTS     = models/yolov4-tiny.weights
YOLO_NAMES       = models/coco.names
YOLO_CONFIDENCE  = 0.4   # min confidence to accept a detection
YOLO_THRESHOLD   = 0.4   # NMS overlap threshold
SUSPICIOUS_OBJECTS = ["backpack", "handbag", "suitcase", "cell phone", "knife", "scissors"]
```

4. Web Dashboard - How It Works

The dashboard is a Flask web application. Flask is a small Python web framework that turns Python functions into web pages. The dashboard shows a LIVE camera feed in the browser with detection boxes drawn on every frame.

The Big Challenge: Live Video in a Browser

A browser cannot directly use a webcam in Python. So we use a technique called MJPEG streaming (Motion JPEG):

- A background thread captures webcam frames continuously
- Each frame is processed (face + object detection)
- The processed frame is encoded into JPEG image bytes
- Flask sends these JPEG images one after another in an endless stream
- The browser displays them like a live video (30 small images per second look like video)

Key: Threading

The camera loop runs in a separate "thread" (a background worker) so that it runs continuously while Flask waits for browser requests. Without threading, the camera would block the web page and vice versa.

5. Dashboard Video Processor (src/dashboard.py)

Class: VideoProcessor

This class runs the whole video pipeline in a background thread. It has these attributes:

- frame - the latest processed frame ready to stream
- faces - list of the faces detected in the current frame
- objects - list of objects detected in the current frame
- running - flag used to stop the thread
- thread - the background threading.Thread object
- recognizer, object_detector, email_alert, logger - the modules from before

Methods

start() / stop()

Starts the background thread and stops it safely.

_capture_loop(self)

The main loop running inside the thread. Step by step:

- Open the webcam (same as before)
- Loop forever while running is True:
- -> read one frame
- -> recognize faces and draw boxes on the frame
- -> every 3rd frame, run object detection (saves CPU)
- -> draw object boxes on the frame
- -> if a known face matched AND cooldown passed AND email enabled: save snapshot, send email, log it
- -> draw an info bar (uptime, counts)
- -> store the finished frame in self.frame

get_frame_jpeg(self)

Takes the latest frame, encodes it into JPEG bytes using cv2.imencode(".jpg", frame), and returns the bytes. Flask uses this to serve the video stream.

6. Flask Web App (app.py)

app.py is the entry point. Running it starts the camera thread and the web server. It defines all the web pages (routes). Here is each route:

Route	Kind	What it does
/	Page	Main dashboard: live feed + stats + current detections
/video_feed	Stream	Endless JPEG video stream (MJPEG)
/alerts	Page	Table of all detection history
/known_faces	Page	List of registered faces with photos
/snapshot/<id>	Image	Serves the photo of a known face
/api/stats	JSON	Statistics for programs/APIs
/api/streaming_status	JSON	Email-on/off and pipeline status

Important Code Details

The video feed generator

```
def generate():
    while True:
        jpeg = processor.get_frame_jpeg()
        if jpeg:
            yield (b'--frame\r\n'
                   b'Content-Type: image/jpeg\r\n\r\n' + jpeg + b'\r\n')
return Response(generate(), mimetype='multipart/x-mixed-replace; boundary=frame')
```

This "generator" loop never ends. It keeps yielding JPEG images. The browser shows them one after another as a live video. Each yield ends with the marker --frame so the browser knows where one picture ends and the next begins.

Server Startup (bottom of app.py)

```
if __name__ == '__main__':
    processor.start()                        # start camera thread
    try:
        app.run(host='0.0.0.0', port=5000)     # start web server
    finally:
        processor.stop()                       # stop camera on exit
```

host="0.0.0.0" means ANY device on the same Wi-Fi network can open the dashboard. port=5000 is the door number. use_reloader=False prevents Flask from restarting twice (which would open two cameras).

7. HTML Templates (templates/)

Flask uses HTML "templates" for pages. They live in the templates/ folder. They use Jinja2 syntax ({{ variable }}) to fill in data from Python.

base.html

The layout shared by all pages. Contains the top navigation bar (Live Feed, Alerts, Known Faces) and all the CSS styling (colors, tables, badges). Other templates "extend" this one, so the design stays consistent.

index.html (Live Feed page)

Shows:

- Five stat boxes: Known Faces, Total Detections, Matches, Unknown, Email status
- The live video: <img src="/video_feed"> - the browser loads this URL and streams video
- "Currently Detected" card: lists names and objects seen right now with color badges

alerts.html (Alerts page)

A table of the last 100 detections from the database: ID, timestamp, name, confidence, camera, and a MATCH/UNKNOWN badge.

known_faces.html (Known Faces page)

A table of everyone registered in the database: ID, name, photo (loaded via /snapshot/<id>), and the registration date.

8. How All Modules Work Together

```
app.py (Flask server + routes)
  |
  |-- uses --> src/dashboard.py (VideoProcessor thread)
  |              |-- uses --> src/face_recognizer.py (faces)
  |              |-- uses --> src/object_detector.py (YOLO objects)
  |              |-- uses --> src/email_alert.py (emails)
  |              |-- uses --> src/detection_logger.py (CSV logs)
  |
  |-- uses --> src/database.py (SQLite: known faces + detections)
  |
  +-- renders --> templates/ (HTML pages)
  |
  +-- streams --> /video_feed (MJPEG)

Browser <---> Flask <---> VideoProcessor thread <---> webcam
                     |---> YOLO draws boxes
                     |---> faces drawn
                     |---> email when match
```

9. How to Run the New Features

Run the Dashboard

```
cd ~/Desktop/Desktop/major_project
/usr/bin/python3 app.py
```

Then open your browser and go to: http://localhost:5000 (or http://127.0.0.1:5000)

To open the dashboard from YOUR PHONE (same Wi-Fi): find your computer IP with `ip addr` or `hostname -I`, then visit http://YOUR_IP:5000

Run the Old Terminal Version (still works)

```
/usr/bin/python3 -m src.surveillance
```

Test Object Detection Alone

```
/usr/bin/python3 -c "from src.object_detector import ObjectDetector; print(len(ObjectDetector().classes))"
```

10. What You See on Screen

In the Browser Dashboard

- Top-left: camera uptime and counts bar (drawn on the video)
- Green box = unknown face
- Red box + "MATCH: name" = recognized person
- Blue box = normal object (person, chair...)
- Red box + yellow badge = SUSPICIOUS object (backpack, bag, phone)
- Below the video: badges listing current faces and objects

Dashboard Pages

- / -> live feed + statistics
- /alerts -> full detection history table
- /known_faces -> registered people with photos

11. Files in This Update

Folder	File	What it is
src/	object_detector.py	YOLO object detection module (NEW)
src/	dashboard.py	VideoProcessor background thread (NEW)
root	app.py	Flask web application (NEW)
templates/	base.html	Shared page layout (NEW)
templates/	index.html	Live feed page (NEW)
templates/	alerts.html	Alerts history page (NEW)
templates/	known_faces.html	Known faces page (NEW)
models/	yolov4-tiny.cfg	YOLO config file (NEW)
models/	yolov4-tiny.weights	YOLO weights ~24MB (NEW)
models/	coco.names	80 class names (NEW)
config/	settings.py	Added YOLO + dashboard settings
src/	database.py	Fixed thread-safety (check_same_thread=False + lock)

12. Troubleshooting

"Address already in use" when running app.py

Port 5000 is busy (maybe another server). Close it or change the port in app.py.

```
pkill -f "app.py"
# or change port=5000 to port=5001 in app.py
```

YOLO detects nothing

Make sure an object is actually in view (a person, a bag). Detection confidence must be above 0.4.

Lower YOLO_CONFIDENCE in settings.py to 0.25 for more detections (but more false positives).

Dashboard shows black/empty video

The webcam may be busy. Check no other program uses it, then restart app.py with Ctrl+C and rerun.

"No module named flask"

Install it for the correct Python:

```
/usr/bin/python3 -m pip install flask
```

YOLO files missing

Put yolov4-tiny.cfg, yolov4-tiny.weights and coco.names inside the models/ folder.